9

PART 3: PRACTICE AND IMPLICATIONS

Brownfield development

How do you recover a usable specification from a system that never had one written down?

A developer at a retail bank picks up a change request that sounds routine: give customers a same-day grace period before an overdraft fee posts. The bank's core ledger has run since the late 1980s, written in COBOL by architects nobody currently on staff has ever met. She opens the module that calculates fees and starts tracing where the number actually gets decided.

AcctMaintenance calls BalanceEngine. BalanceEngine calls FeeSchedule. FeeSchedule calls a routine nobody has touched since sometime in the early 2000s, and three screens down she finds the line she's looking for: `IF ACCT-STATUS-CD = '07' THEN WAIVE-FEE`. Seven. Not documented in the code, not in the requirements binder from the system's early years, not in anyone's memory she's asked so far.

This is **brownfield development**: building on a system that is already live, already trusted with somebody's money, and already harder to read than the people who wrote it ever intended. Most software careers happen here, on systems built years or decades before, not on the blank slate every conference talk assumes. The code is the one thing that reliably survived. Whatever the business meant by it did not.

CHAPTER PROMISE

By the end of this chapter, you can treat an undocumented system as a specification waiting to be read, not only a codebase waiting to be replaced, and use AI to surface what its behavior implies about the business without asking it to decide anything on its own.

Reading time: 13 minutes

Where the missing knowledge actually goes

Code tells you how a system behaves. It rarely tells you why, and the gap between those two things is where the developer above will spend most of her week. She can find status code 7 in an afternoon; grep is good at that. Finding out what it meant, and whether waiving the fee for that status was ever a considered decision or just the behavior a fix from twenty years ago happened to produce, takes days of asking people who may no longer work there.

That search is the real cost of a missing specification, and it compounds. A requirement that would take an afternoon to confirm against a written use case instead takes a week of tracing calls, reading commit history nobody wrote useful messages for, and interviewing whoever has been there longest. Multiply that by every change request a legacy system receives over a decade, and the tax adds up. By the bank's own account, reported in Appendix A, adding a feature took six to eight weeks, and the tracing came before any of the building.

It gets worse than slow. The bank had tried, once before this chapter's scene, to replace the COBOL ledger outright: translate it line for line into a modern language, keep the behavior, retire the mainframe. Partway through, the team building the replacement noticed it didn't match the original in the edge cases, rounding here, an ordering difference there, and couldn't tell whether the mismatches were bugs to fix or business rules to preserve. Nobody left at the bank could say for certain. The project stalled on a question no amount of translation could answer: translation preserves syntax, not intent, and intent was the part nobody had written down.

Business knowledge that lives only in code slowly dissolves into implementation detail, until the only thing left of a rule is its side effect.

Reading a system as a specification, not a debugging problem

Specification recovery is the practice this chapter is about: reconstructing a system's vision, requirements, use cases, and domain model from what its code and its people currently do, instead of writing those layers from a blank page the way the rest of this book has, so far, assumed you could. It runs the specification-driven method in reverse. Rather than starting from a decision and generating code, you start from code and work backward to the decision that, at some point, someone made.

Martin Fowler already named a version of this discipline, applied to architecture rather than knowledge. He called it the Strangler Fig pattern: intercept calls to one piece of old functionality at a time, route them to a new implementation, and retire the old piece once nothing depends on it anymore. It's an established, widely used way to replace software gradually instead of all at once, and it works because it never asks a team to understand the whole system before touching any of it.

Specification recovery borrows that discipline and points it one layer earlier. Instead of replacing running code piece by piece, you recover the knowledge behind one piece of it, validate that knowledge with someone who can vouch for it, and only then decide what should replace the code. Call this chapter's version an extension of Fowler's logic, not a restatement of it: his pattern strangles code without ever requiring anyone to understand what it was for, while this one only works if someone does.

> **Code can be replaced by a team that never understood it. A specification can only be recovered by people who do, which makes recovery the slower half of the work and the half that decides what survives the migration.**

Recover, validate, improve, then generate

A straight line-for-line migration asks one question: does the new code do what the old code did? Specification recovery asks a different one first, and defers the migration until it has an answer.

1. Recover. Point an AI assistant at one module, one workflow, or one integration at a time, and ask it to describe, not to fix. Entities, business rules, validation logic, the shape of a use case, all of it read out of the code as it stands today.
2. Draft the specification layers from that description, flagging anything that looks like suspicious complexity, duplicated logic, a magic number, a rule with no visible reason, rather than resolving it. This is where AI's role stays archaeological: it surfaces what's odd, it does not decide what the odd thing was for.
3. Validate every recovered rule with a person who has standing to confirm or dispute it, an operations lead, a compliance officer, whoever has lived with the system's actual behavior. Some rules will be confirmed instantly. Others will start an argument, and that argument is the point.
4. Only once a rule is confirmed as intentional, or corrected as a mistake nobody meant to keep, generate the new implementation against it. Improve the architecture at this stage, not the business meaning; that decision was already made in step three.

Recovering "process a withdrawal" from the bank's ledger

Back to the developer and her overdraft fee. She and an AI assistant work through FeeSchedule and the two modules that call it, and instead of asking for a fix, she asks for a description: what does this code actually do, rule by rule, before anyone decides whether it's right.

The first two rules come back uncontroversial. A withdrawal that would take the available balance below the account's minimum is declined unless an overdraft line is attached; everyone who's touched the ledger recognizes that one on sight. Status code 7 takes longer, but an operations lead who has supported the system for over a decade remembers it: a hardship flag, set by a case worker for a customer in an approved financial-hardship program, waiving the overdraft fee for as long as the flag is active. Confirmed, and now written down for the first time.

The third rule is where the recovery stops being tidy. The code always posts same-day deposits before same-day withdrawals, regardless of which one the customer submitted first. A compliance analyst reviewing the draft specification flags it immediately: transactions should process in the order they arrive, and this looks like a defect that happens to favor the customer. The operations lead disagrees just as fast. That ordering, she says, has kept thousands of accounts out of overdraft for as long as she's worked there; change it, and the bank will hear about it from customers before the week is out.

Neither of them is wrong about what the code does. They disagree about what it means, and that is not a question AI can settle by reading more code. It goes into the recovered specification as a flagged decision, not a resolved one.

> **Recovered use case: process a withdrawal (draft, in validation)**
>
> Primary actor: retail account holder, via the ledger's nightly batch.
>
> BR-1: Decline if available balance minus the withdrawal falls below the account minimum, unless an overdraft line is attached. Confirmed.
>
> BR-2: Waive the overdraft fee when account status is 07 (hardship flag, case-worker set). Confirmed by operations; undocumented anywhere else.
>
> BR-3 [DISPUTED]: same-day deposits post before same-day withdrawals, regardless of submission order. Compliance reads this as a defect. Operations reads it as fifteen years of protecting customers from overdraft fees. Recovered as-is. Decision owner: head of retail operations.

What this work costs when a specification was never missing

Riverside Clinics has no equivalent of status code 7, and the difference is worth sitting with. Say the clinic group needs to move its booking system off its current stack ten years from now, to whatever platform has replaced it by then. Nobody will need to reverse-engineer why a slot only counts as available when the doctor works at that location, or where the 90-day booking horizon came from, because Chapter 5 wrote both down as REQ-07, in the same document as the use case that applies them, and Chapters 6 through 8 kept it current as the system grew.

The work in that future migration looks nothing like this chapter's. Someone opens the versioned specification, checks it against what the system currently does, updates what's changed since the last version, and hands the result to AI for the new stack. No archaeology, because nothing was buried in the first place. No dispute between operations and compliance over what a rule was always supposed to mean, because the rule was written down by the people who made the decision, not inferred years later from its side effects.

THE DIFFERENCE IN ONE LINE

The bank is paying, decades late, for a specification it never wrote. Riverside is spending a little now so that a future migration would have nothing to recover, only something to carry forward.

Where recovery goes wrong

▲ **WARNING**

Everything AI recovers from a legacy system is a hypothesis about intent, not a fact about it, until a person with standing to know confirms or corrects it. Treating a recovered rule as settled the moment it's written down skips the one step that makes recovery different from guessing with extra steps.

Trusting a recovered rule without validating it	A team builds a new implementation against an AI-inferred rule, ships it, and later learns the rule was an edge-case workaround someone patched in once, not a general policy anyone intended to keep.
Treating recovery as a one-time project	A team recovers a specification, generates a clean new system from it, and never updates the specification again. Five years and a dozen unrecorded changes later, it is exactly as unreliable as the code it replaced.

Field guide: a specification-recovery checklist

- ☐ Has every recovered business rule been traced to a person who can confirm or dispute it, not only to the line of code it lives in?
- ☐ Is anything AI flagged as suspicious logged for a human decision, rather than quietly changed or quietly kept?
- ☐ Does the recovered specification distinguish behavior confirmed intentional, behavior confirmed accidental, and behavior nobody has judged yet?
- ☐ Would a stakeholder who has never read the original code recognize the recovered use case as their own process?

Run this before generating any new implementation from a recovered specification. An unconfirmed rule that reaches production is a bug wearing the credibility of a written specification.

What to remember

- Most software careers are spent maintaining systems that already exist, not building ones that don't; brownfield work, not greenfield, is the default.
- Code records how a system behaves. It does not reliably record why, and business knowledge stored only in code eventually loses its meaning entirely.
- AI's most useful role in a legacy system is archaeological: surfacing suspicious complexity for a human to judge, not rewriting anything on its own authority.
- A recovered rule is a hypothesis until someone with standing confirms it. Two people can read the same behavior and disagree about what it was always supposed to mean, and that disagreement belongs to the business, not to the code.
- Recover, validate, improve, then generate, in that order. Skipping straight to generation just automates the guessing a manual rewrite would have done anyway.

QUESTIONS FOR YOUR TEAM

- Which system your team maintains has the most undocumented business rules buried in its code, and who is the one person left who could still confirm what any of them mean?
- Which of those rules were ever written down somewhere a successor could find them?

ONE ACTION TO TAKE NEXT

Pick one module of a legacy system your team maintains. Ask an AI assistant to describe, not fix, what it does, rule by rule, then take that description to the one person most likely to confirm or dispute it.

TRANSITION. Recovering a specification answers what a legacy system does. It doesn't answer what happens to the engineers who used to be the only ones who knew. The next chapter turns from the systems to the people who build and maintain them.

Brownfield development ends here.